

Rapport de veille technologique frontend

Vue

Vue est un framework JavaScript pour construire des interfaces à partir de composants. Ses templates prolongent le HTML, et son système de réactivité met à jour l'affichage lorsque l'état change. Un fichier .vue réunit le template, la logique et le style d'un composant : pour le Kanban, les colonnes, les listes de tâches et les fenêtres d'édition deviennent des composants distincts. [1]

La Composition API regroupe l'état et les fonctions d'une même fonctionnalité, et les composables permettent de partager ce comportement entre plusieurs composants. La syntaxe `<script setup>` réduit le code répétitif et prend en charge le typage des props et des événements. [2] Vue Router et Pinia sont des solutions officielles pour la navigation et l'état partagé, ce qui évite de choisir chaque brique soi-même.

En revanche, il faut comprendre comment interagissent `ref`, `computed` et `watch`, et la coexistence de deux styles d'API impose de fixer une convention d'équipe. Vue ne conçoit pas non plus l'architecture à notre place : l'accès aux données reste à organiser. Enfin, Vue échappe le texte interpolé, mais afficher du HTML non fiable avec `v-html` reste dangereux. [3] Je retiens donc Vue pour une interface à composants, avec une équipe qui s'accorde sur ses conventions dès le départ.

Sources

- 1 Vue : introduction et composants monofichiers.
<https://vuejs.org/guide/introduction.html>
- 2 Vue : FAQ de la Composition API et syntaxe `<script setup>`.
<https://vuejs.org/guide/extras/composition-api-faq.html>
- 3 Vue : bonnes pratiques de sécurité.
<https://vuejs.org/guide/best-practices/security.html>

Angular

Angular est un framework maintenu par Google. Il associe des composants TypeScript à des templates HTML et fournit un ensemble intégré : routeur, formulaires, client HTTP et outillage en ligne de commande. Son injection de dépendances fournit des services partagés aux composants ; pour le Kanban, un service centraliserait les requêtes vers l'API. [1]

Son principal atout est la cohérence : les besoins courants ont une solution officielle. Les formulaires réactifs donnent un modèle explicite pour la validation, et `HttpClient` propose des intercepteurs, utiles pour ajouter le jeton d'authentification à chaque requête. [2] Les Signals suivent les dépendances de l'état pour mettre à jour l'interface de façon ciblée. [3]

Pour un petit projet en temps limité, l'inconvénient serait la quantité de notions à apprendre ensemble : métadonnées des composants, injection, formulaires, routage et flux asynchrones. C'est une appréciation liée à ce projet, et non une preuve qu'Angular serait toujours lent ou difficile. Ses conventions prennent de la valeur lorsqu'une équipe les connaît déjà ou maintient plusieurs applications. Je n'ai pas développé avec Angular : cette analyse repose sur sa documentation officielle. Je le considérerais sérieusement pour une équipe qui privilégie un cadre complet.

Sources

- 1 Angular : vue d'ensemble et injection de dépendances.
<https://angular.dev/overview>
- 2 Angular : formulaires réactifs et client HTTP.
<https://angular.dev/guide/forms/reactive-forms>
- 3 Angular : Signals.
<https://angular.dev/guide/signals>

React

React est une librairie pour construire des interfaces, plutôt qu'un framework complet. Les composants sont des fonctions JavaScript écrites en JSX, qui mêle le balisage et les expressions JavaScript. L'état et les props décrivent ce qui doit s'afficher ; un tableau Kanban se découpe en composants de colonne, de tâche et de formulaire. [1]

Ses atouts sont la réutilisation des composants et un flux de données explicite : remonter l'état partagé dans un parent commun permet de coordonner les parties du tableau. [1]

React laisse aussi le choix des outils : sa documentation recommande un framework pour une nouvelle application, tout en permettant de partir de zéro, par exemple avec Vite. [2] C'est enfin la plus utilisée des trois technologies, par 46,9 % des développeurs professionnels interrogés en 2025. [3]

Cette liberté crée en contrepartie des décisions : le routage, la récupération des données et la gestion de l'état demandent des choix supplémentaires. Les Effects sont un autre point d'attention, car un Effect inutile provoque des rendus en trop et des problèmes de synchronisation. [4] Comme avec Vue, insérer du HTML non fiable avec dangerouslySetInnerHTML expose à des failles XSS. React répondrait aux besoins du Kanban, à condition de fixer ces conventions avant de commencer.

Sources

- 1 React : penser en React (composants, état et props).
<https://react.dev/learn/thinking-in-react>
- 2 React : créer une application React.
<https://react.dev/learn/creating-a-react-app>
- 3 Stack Overflow : Developer Survey 2025, frameworks web.
<https://survey.stackoverflow.co/2025/>
- 4 React : vous n'avez peut-être pas besoin d'un Effect.
<https://react.dev/learn/you-might-not-need-an-effect>

Pourquoi j'ai choisi Vue

J'ai choisi Vue pour réaliser ce Kanban en me plaçant dans le rôle d'un lead dev qui démarre un produit avec une petite équipe : deux développeurs juniors et un intégrateur HTML/CSS. Les templates de Vue restent proches du HTML qu'ils connaissent, et le routeur comme le store sont officiels : il y a moins de choix à imposer au départ.

L'application utilise Vue 3.5 avec TypeScript, Vite, Vue Router et Pinia. Les composants passent par un module d'accès aux données, dans src/api : remplacer la source de données toucherait cinq fichiers et aucun composant. [2] PrimeVue fournit les composants d'interface et Tailwind la mise en page ; ce sont des choix du projet, pas des fonctionnalités de Vue.

L'apprentissage a surtout porté sur la réactivité, avec le `.value` à écrire dans le script mais pas dans le template, et sur les cas limites du réseau : une réponse tardive après une déconnexion, ou un réordonnancement des colonnes qui n'est pas atomique côté API. J'ai livré 45 points sur 86 en quatre jours, pour environ 40 heures estimées après coup. [2]

Côté sobriété, le build pèse 121 ko de JavaScript compressé, dont 46 % viennent de PrimeVue, et Lighthouse donne 100 en performance. [1] [3] Mon choix reste lié à ce contexte : je n'ai pas construit d'application équivalente en Angular ou en React. Pour une équipe déjà formée à l'un d'eux, il serait logique de le garder.

Sources et éléments du projet

- 1 Vue : bonnes pratiques de performance.
<https://vuejs.org/guide/best-practices/performance.html>
- 2 Projet : package.json, dossier src/api, historique Git et GitHub Projects « veille-techno-frontend ».
<https://github.com/oussema-fatnassi/veille-techno-frontend>
- 3 Mesures : npm run build et Lighthouse desktop, 4 octobre 2026.

Sources consultées le 5 octobre 2026. Les documentations citées concernent Vue 3, Angular et React 19 ; seule l'application Vue a été réalisée.